

Understand Kamba — Project Documentation

English and Kikamba translation, Kikamba speech recognition, and read-aloud playback, running entirely on your own machine.

Last updated: 29 September 2026

1. Overview

Understand Kamba is a web app backed by a Python API. It lets you:

- translate English into Kikamba and Kikamba into English (and Kikamba into Swahili through the API),
- speak or upload Kikamba audio and get it transcribed to text,
- hear Kikamba text read aloud,
- rate, copy, share and save translations.

All models run locally. No paid API or account is needed with the default settings, and no text or audio leaves the machine running the backend.

Status: working prototype. Every feature in the interface is backed by a real model, but translation accuracy has not yet been checked by a Kikamba speaker, and the voice is a Swahili voice reading Kikamba (see section 8). Treat outputs as drafts, not authoritative translations.

2. Features at a glance

Feature	How it works	Quality today
English to Kikamba, Kikamba to English	Meta NLLB-200 (distilled 600M)	Unverified by a Kikamba speaker. Kamba is low-resource in NLLB and some output shows Kikuyu-style grammar.
Kikamba speech to text	FarmerlineML/w2v-bert-2.0_kamba	About 30% word error rate and 7% character error rate on its publisher's own test set.
Read aloud	Meta MMS Swahili voice (facebook/mms-tts-swh)	Clear and intelligible, but with a Swahili accent; ĩ and ũ are read as i and u.
Usage stats and ratings	Counted by the backend, stored in data/	Real numbers.
Copy, share, save	In the browser	Saved translations stay in that browser only.
Dialect selection	Label only	All four dialects use the same models, so the output does not change.

3. How it fits together

The system has two parts that run side by side:

- **Backend** (app/): a FastAPI server on port 8000. It loads the models, serves the API, and keeps usage stats in data/.
- **Web app** (web/): a Next.js site on port 3000. It runs in the browser and calls the backend over HTTP.

What happens for each action:

Action in the web app	Backend call	Model used
Translate	POST /translate	NLLB-200
Speak or Upload (Kikamba mode)	POST /transcribe?translate=false, then POST /translate	w2v-BERT Kamba, then NLLB-200
Play	POST /synthesize	MMS Swahili voice
Thumbs up or down	POST /feedback	none
Header stats	GET /stats	none

Each model loads the first time it is needed, which takes 10 to 30 seconds. After that, requests take a few seconds on a typical CPU.

Audio uploads are converted to 16 kHz mono WAV with ffmpeg before transcription and are trimmed to the first 60 seconds. Temporary audio files are deleted as soon as each request finishes.

An older static interface in frontend/ is also served by the backend at <http://localhost:8000/>. It offers file upload, video-link transcription and Kikamba-to-Swahili/English text translation.

4. Getting started

4.1 Requirements

- Python 3.12 (3.10 or newer should work)
- Node.js 20 or newer, with npm
- About 8 GB of free disk space for the models, and 8 GB of RAM or more
- An internet connection the first time each model is used, to download it

A separate ffmpeg install is not needed: the backend uses ffmpeg from your PATH if there is one, and otherwise the copy bundled with the imageio-ffmpeg Python package.

4.2 Install

From the project folder:

```
python -m venv venv
venv\Scripts\activate      # Windows
# source venv/bin/activate  # macOS / Linux
pip install -r requirements.txt

cd web
npm install
```

4.3 Configure

Copy `.env.example` to `.env` in the project folder. The defaults work as they are; see section 6 for every setting.

To point the web app at a backend somewhere other than `http://localhost:8000`, copy `web/.env.local.example` to `web/.env.local` and set `NEXT_PUBLIC_API_BASE_URL`.

4.4 Run

Start the backend (from the project folder, with the virtual environment's Python):

```
venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000 --env-file .env
```

The `--env-file` flag matters: the app does not load `.env` by itself.

Start the web app in a second terminal:

```
cd web
npm run dev
```

Then open `http://localhost:3000`.

4.5 First run

The first translation, transcription and playback each download and load a model. Sizes: NLLB-200 about 4.7 GB, w2v-BERT Kamba about 2.3 GB, MMS Swahili voice about 140 MB. Models are cached in `~/.cache/huggingface/hub` and are not downloaded again.

5. Using the web app

Choose a direction. The switch at the top right of the card selects English to Kikamba or Kikamba to English. Switching swaps the two sides, so your last translation carries over.

Translate text. Type in the left box and press Translate, or Ctrl+Enter (Cmd+Enter on a Mac). Input is limited to 500 characters. In English mode, the Sample button fills in an example sentence.

Speak Kikamba. In Kikamba to English mode, press Speak, talk, then press Stop (recording stops by itself after 60 seconds). Or press Upload and choose an audio or video file. The transcript appears in the left box and is translated automatically. The browser asks for microphone permission the first time; recording only works on localhost or over HTTPS.

Listen. In English to Kikamba mode, press the play button under the translation. The first press generates the audio and draws its waveform; later presses replay it.

Rate. Thumbs up or down rates the current translation once. Ratings update the "% rated helpful" figure in the header and are logged with the sentence for later review.

Copy, share, save. Copy puts the translation on the clipboard. Share opens your device's share menu, or copies both texts on a computer. Save adds the pair to the Saved translations list below the card; it is kept in this browser only and can be removed with the bin icon.

Dialect. The cards under Dialects & voices change the title and tag shown with the translation. The translation itself is the same for every dialect.

Record your voice (/contribute). Kikamba speakers can help build a real Kamba voice. The page is linked from the home page ("Record your voice") and from the note under the play button. Contributors read a short explanation of how recordings are used, choose their dialect (gender and age are optional), confirm they are 18+ and fluent, and agree. They then see one Kikamba sentence at a time: record (up to 20 seconds), listen back, then submit, re-record or skip. The browser remembers the anonymous contributor ID and progress; "Ask to delete my recordings" opens a GitHub issue with that ID.

6. Configuration reference

Backend settings are read from environment variables, normally supplied through `.env`.

Variable	Default	Purpose
TRANSLATION_BACKEND	nllb	nllb (local, free) or anthropic (Anthropic API, paid per request)
NLLB_MODEL_ID	facebook/nllb-200-distilled-600M	NLLB model to load. facebook/nllb-200-distilled-1.3B is larger and usually more accurate.
NLLB_NUM_BEAMS	5	Candidate translations compared per sentence. Higher is slightly better and slower.
ANTHROPIC_API_KEY	(empty)	Required only when TRANSLATION_BACKEND=anthropic
TRANSLATION_MODEL	(empty)	Claude model ID, required only when TRANSLATION_BACKEND=anthropic
MAX_TRANSLATE_CHARS	500	Longest text accepted. Keep in step with MAX_TRANSLATE_CHARS in web/src/lib/api.ts.
ASR_MODEL_ID	FarmerlineML/w2v-bert-2.0_kamba	Speech recognition model. facebook/mms-1b-all also works, but its Kamba support is unconfirmed.
ASR_TARGET_LANG	kam	Language adapter, used only with MMS models
MAX_AUDIO_SECONDS	60	Audio longer than this is trimmed. Keep in step with MAX_RECORDING_SECONDS in web/src/lib/api.ts.
SAMPLE_RATE	16000	Sample rate audio is converted to before transcription
TTS_MODEL_ID	facebook/mms-tts-swh	Voice model; any MMS/VITS text-to-speech model works
UPLOAD_MAX_MB	25	Largest audio upload accepted
TEMP_DIR	system temp folder/kam-backend	Where uploads are converted; files are deleted after each request
DATA_DIR	data/ in the project folder	Where stats.json, feedback.jsonl and voice recordings (data/voice/) are kept

Variable	Default	Purpose
VOICE_DATASET_REPO	(empty)	Hugging Face dataset that contributed recordings are pushed to every 5 minutes, e.g. lawmaluki/kamba-voice-recordings. Empty keeps recordings local only. Needs HF_TOKEN with write access.
VOICE_MAX_SECONDS	20	Longest recording accepted
VOICE_UPLOAD_MAX_MB	5	Largest recording upload accepted
VOICE_UPLOADS_PER_HOUR	120	Recordings accepted per connection per hour
NLLB_GPU_MODEL_ID	(empty)	Hugging Face Space only (space/server.py): translation model run on the ZeroGPU GPU, e.g. facebook/nllb-200-3.3B. NLLB_MODEL_ID becomes the CPU fallback.

Web app setting (web/.env.local):

Variable	Default	Purpose
NEXT_PUBLIC_API_BASE_URL	http://localhost:8000	Address of the backend

7. API reference

All endpoints accept and return JSON unless noted. Errors return a JSON body of the form {"detail": "..."}.

Interactive docs are available at <http://localhost:8000/docs> while the backend is running.

POST /translate

Translates text.

Request:

```
{ "text": "Good morning, my friend.", "direction": "en_to_kam" }
```

direction is one of: en_to_kam, kam_to_en, or kam_to_sw_en (Kikamba to both Swahili and English; the default).

Response:

```
{ "text": "...", "translation_kam": "...", "translation_en": null, "translation_sw": null,
  "model": "facebook/nllb-200-3.3B" }
```

Only the fields for the requested direction are filled in. model names the model that produced the translation; on the Hugging Face Space it shows whether the GPU model or the CPU fallback answered.

Errors: 400 if the text is empty; 422 if it is longer than MAX_TRANSLATE_CHARS; 503 if the Anthropic backend is selected but not configured; 500 if the model fails.

POST /transcribe

Transcribes Kikamba speech. Send the audio as multipart form data in a field named file. Any format ffmpeg reads works, including webm, m4a, mp3, wav and video files.

Query parameter translate (default true): when true, the transcript is also translated to Swahili and English.

Response:

```
{ "transcript": "...", "translation_sw": "...", "translation_en": "..." }
```

Errors: 413 if the file is larger than UPLOAD_MAX_MB; 500 if conversion or transcription fails.

POST /transcribe-url

Same as /transcribe, but downloads the audio from a video or social-media link using yt-dlp (files up to 200 MB, 120-second download limit).

```
{ "url": "https://..." }
```

Errors: 400 if the link cannot be downloaded; 500 if transcription fails.

POST /synthesize

Reads Kikamba text aloud. Returns audio/wav (16 kHz mono), not JSON.

```
{ "text": "Ūnĩ mūseo, mūnyanyawa." }
```

Errors: 422 if the text is empty or too long; 500 if generation fails.

GET /stats

```
{ "translations": 42, "ratings_up": 10, "ratings_down": 3 }
```

translations counts successful /translate calls.

POST /feedback

Records a rating and returns the updated stats.

```
{ "rating": "up", "direction": "en_to_kam", "text": "source text", "translation": "translated text" }
```

rating is up or down. Errors: 422 for any other value.

GET /voice/prompts

The Kikamba sentences contributors read aloud (1,044 from FLORES-200), plus the current consent version and recording limit.

```
{ "prompts": [{ "id": "flores-dev-12", "text": "..." }], "consent_version": "2026-09-30", "max_seconds": 20 }
```

POST /voice/recordings

Stores one contributed recording. Multipart form data:

Field	Required	Notes
file	yes	The recording (any format ffmpeg reads, such as the browser's webm)
prompt_id	yes	The id of the sentence read, from /voice/prompts
speaker_id	yes	The contributor's anonymous UUID
dialect	yes	machakos, kitui, makueni or other
consent	yes	Must be true
gender	no	female, male, other, or empty
age_range	no	18-29, 30-44, 45-59, 60+, or empty

The recording is converted to 16 kHz mono WAV and stored with a metadata row (the sentence text is taken from the server's list, not the client). Response: {"ok": true, "duration_seconds": 5.7}.

Errors: 413 if the upload is over VOICE_UPLOAD_MAX_MB; 422 with a readable message if consent is missing, the details are invalid, or the recording is under 1 second, over VOICE_MAX_SECONDS or silent; 429 past VOICE_UPLOADS_PER_HOUR.

GET /health

Returns {"status": "ok"}.

8. Models and quality

Translation: NLLB-200

Meta's No Language Left Behind model includes Kamba (code kam_Latn), but Kamba had very little training data. The backend improves its output in three ways:

- each sentence is translated separately, because NLLB was trained on single sentences and drops sentences from longer paragraphs;
- five candidate translations are compared (beam search) instead of taking the first;
- output length is capped relative to the input, and repeated phrases are blocked, to stop runaway or invented text.

Known issue: some words appear to be wrong. For example, "Did you sleep well?" came out as "We nĩwasomie nesa?", where nĩwasomie may mean "did you read". A Kikamba speaker should review output before it is relied on.

Model sizes, scored on 100 FLORES-200 sentences, Kikamba to English (chrF++, higher is better): 600M 34.0, distilled 1.3B 37.5, 3.3B 38.1. English to Kikamba stays around 30 for all three. The local default is the 600M model; the hosted backend runs 3.3B on the Space's GPU with 1.3B as its CPU fallback (see docs/BACKEND-HOSTING-MIGRATION.md, section 10).

Speech recognition: w2v-BERT Kamba

A community fine-tune of Meta's w2v-BERT 2.0 published by Farmerline. Its model card reports a 30% word error rate and 7% character error rate, on a test set that is not described. In testing, words were mostly recognisable but sometimes merged or misspelled.

Voice: MMS Swahili

No working Kikamba voice model exists publicly. The only one found (Musembi/speecht5-tts-kamba) has corrupted weights and outputs silence. The Swahili voice was chosen instead because Swahili spelling is close to Kikamba: when its Kikamba output was transcribed by the Kamba speech model, the text came back almost word for word. Limitations: Swahili accent and intonation, and the vowels *ĩ* and *ũ* are pronounced as plain *i* and *u*.

To switch voices later, set `TTS_MODEL_ID` to any MMS/VITS model, for example a future `facebook/mms-tts-kam`. Meta's Kikuyu (`mms-tts-kik`) and Tharaka (`mms-tts-thk`) voices, both related languages, were also tried; the Kamba speech model misheard 17% and 22% of characters from them, against 8% for Swahili. The `/contribute` page collects the recordings needed to train a genuine Kamba voice.

Measuring accuracy

FLORES-200 (news-style sentences with professional Kamba translations) gives an objective score and was used to compare model sizes above. It does not reflect everyday conversation, so the most useful next step is a second test set: 30 to 50 typical sentences with Kikamba translations written by a speaker. The ratings log in `data/feedback.jsonl` is a good source of sentences that went wrong.

9. Data and privacy

Data	Where it is kept	Lifetime
Uploaded and recorded audio	TEMP_DIR on the backend machine	Deleted when the request finishes
Translation count and rating totals	<code>data/stats.json</code>	Until deleted
Each rating with its source text and translation	<code>data/feedback.jsonl</code>	Until deleted
Saved translations	The browser's <code>localStorage</code>	Until removed in the app or browser data is cleared
Contributed voice recordings and their metadata (sentence, anonymous ID, dialect, optional gender and age range)	<code>data/voice/</code> locally; on the hosted backend, the private dataset <code>lawmaluki/kamba-voice-recordings</code>	Until deleted, for example on the contributor's request
Contributor ID, details and progress	The contributor's browser (<code>localStorage</code>)	Until browser data is cleared
Models	<code>~/.cache/huggingface/hub</code>	Until deleted

Text and audio are not sent to any outside service with the default settings. If `TRANSLATION_BACKEND` is set to `anthropic`, the text being translated is sent to the Anthropic API.

The data/ folder is excluded from git.

10. Project structure

```
understand-kamba/
  app/
    main.py          API endpoints
    config.py         settings from environment variables
    translate.py      translation (NLLB or Anthropic)
    asr.py            speech recognition
    tts.py            read-aloud voice
    audio.py          ffmpeg conversion
    downloader.py     audio from video links (yt-dlp)
    stats.py          translation count and ratings
    voice.py          contributed voice recordings
    data/             kikamba_prompts.tsv (sentences to read aloud)
  web/
    src/app/          home page, /contribute page, global styles, colour tokens
    src/components/translator/
                        StudioCard (state and wiring), StudioHeader,
                        ContentArea, VoiceInput, AudioPlayer,
                        FooterControls, DialectSelector, SavedTranslations
    src/components/contribute/
                        ContributeCta (home-page invitation), ContributeStudio,
                        ConsentStep, RecordingStep
    src/lib/           api.ts (backend calls), dialects.ts, samples.ts,
                        saved.ts, contributor.ts (browser stores),
                        useRecorder.ts (microphone recording)
  space/              Hugging Face Space entry point and config
  frontend/           older static interface served at localhost:8000
  scripts/            model tests, deploy_space.py, build_prompts.py
  docs/              this documentation
  data/              stats and ratings (created on first use)
  .env.example        configuration template
  requirements.txt    Python dependencies
```

The accent colour is butter yellow (#F8E27A), defined as the butter-50 to butter-900 scale in web/src/app/globals.css. Use butter-300 for fills and butter-700 or darker for text on white.

11. Troubleshooting

"Couldn't reach the translation server" in the web app. The backend is not running, or NEXT_PUBLIC_API_BASE_URL points somewhere else. Start the backend (section 4.4) and check <http://localhost:8000/health>.

New endpoints return 404, or settings seem ignored. An old backend process is still running. Stop whatever is using port 8000 and start it again with the command in section 4.4, including --env-file .env.

The first request is very slow. The model is downloading or loading; later requests are faster. Downloads need an internet connection.

"Microphone access was blocked". Allow the microphone for the site in the browser's address bar. Recording requires localhost or HTTPS.

"No speech was recognised in that audio". The recording was silent or too quiet, or not in Kikamba. Record closer to the microphone.

Transcription fails with a file-not-found error. ffmpeg could not be found. Reinstall dependencies with `pip install -r requirements.txt` so imageio-ffmpeg is present.

Stats are not updating. Stats are written to DATA_DIR by a single backend process. Running several backend workers at once is not supported.

12. Known limitations and next steps

- Translation accuracy is unmeasured. Build a speaker-checked test set (section 8).
- No real Kikamba voice. Revisit if Meta or the community publishes one.
- Dialect selection does not change the output; there are no per-dialect models.
- No formal-register option.
- No login, rate limiting or abuse protection, and CORS allows any origin. Tighten these before deploying publicly.
- Stats storage supports a single backend process; move it to a database before scaling.
- No automated tests or deployment pipeline yet.